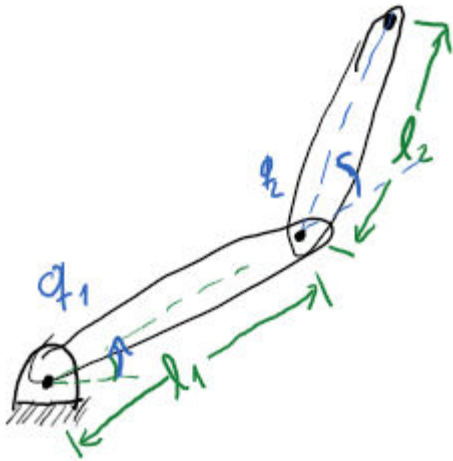


Cinemática Servos

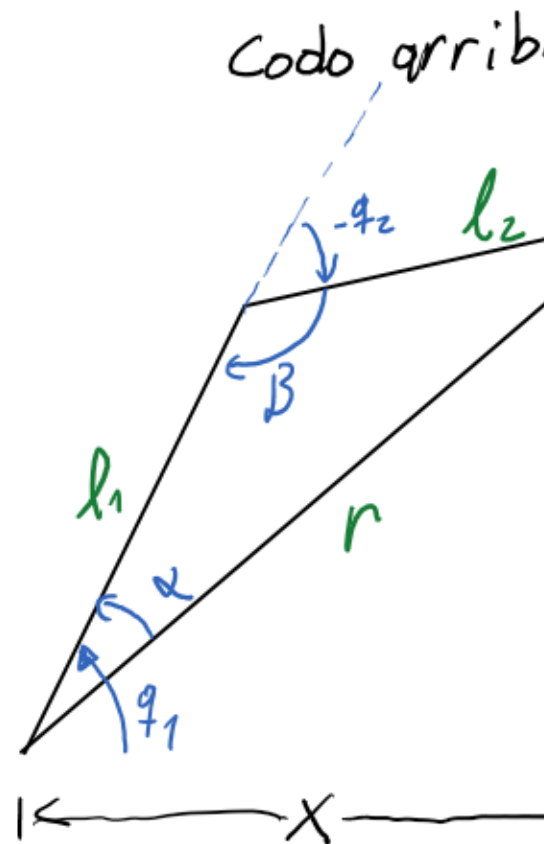
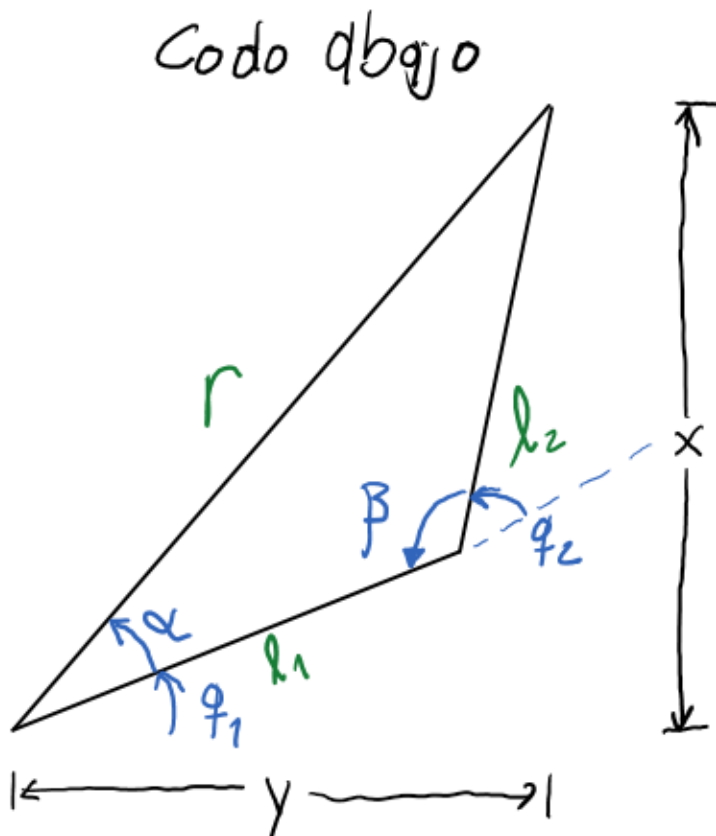
Cinemática inversa para el manipulador

Se tiene un manipulador como el de la figura.



Parámetros del manipulador

Se busca obtener los q_1 y q_2 de forma que el extremo distal del manipulador llegue a x, y . Se observa que el manipulador puede posicionarse de dos formas: codo arriba y codo abajo y se pueden representar de la siguiente forma:



Para ambos casos, por teorema del coseno:

$$\alpha = \arccos\left(\frac{l_1^2 + r^2 - l_2^2}{2 l_1 r}\right)$$

y por el teorema del seno:

$$\beta = \arcsin\left(\frac{r}{l_2} \sin(\alpha)\right)$$

```
% q1=atan(y_pos(n),x_pos(n))-acos((l1^2 + aqrt^2 - l2^2)/(2*20*(sqrt())))  
% q2=asin((r/l2)*sin(alpha))-pi
```

Codo Abajo:

$$\text{atan}(y, x) = q_1 + \alpha$$

$$q_1 = \text{atan}(y, x) - \alpha$$

y

$$\pi = q_2 + \beta$$

$$q_2 = \pi - \beta$$

Codo Arriba:

$$q_1 = \text{atan}(y, x) + \alpha$$

y

$$\pi = -q_2 + \beta$$

$$q_2 = \beta - \pi$$

Obtención de la figura de trébol estilizado:

Se obtuvo de forma empírica la siguiente ecuación para un trébol estilizado de tres hojas que esté inscrito en un cuadrado concéntrico de lado 1:

$$\rho = \frac{4 + \cos(3(\theta - \gamma))}{10}$$

En donde γ es la rotación del trébol.

Se calculan los valores x, y para el trébol y se grafica:

```
clear all, clc, close all  
res = 250;  
theta = linspace(2*pi/res, 2*pi, res);  
gamma = 0;  
rho = (2.5 + cos(3*(theta - gamma)))/0.43;  
[x_pos_pre y_pos_pre] = pol2cart(theta, rho);
```

```

x_pos=x_pos_pre/80;
y_pos=y_pos_pre/80;

x_coord = cos(theta).*((100*cos(3*theta))/43 + 250/43);
y_coord = sin(theta).*((100*cos(3*theta))/43 + 250/43);

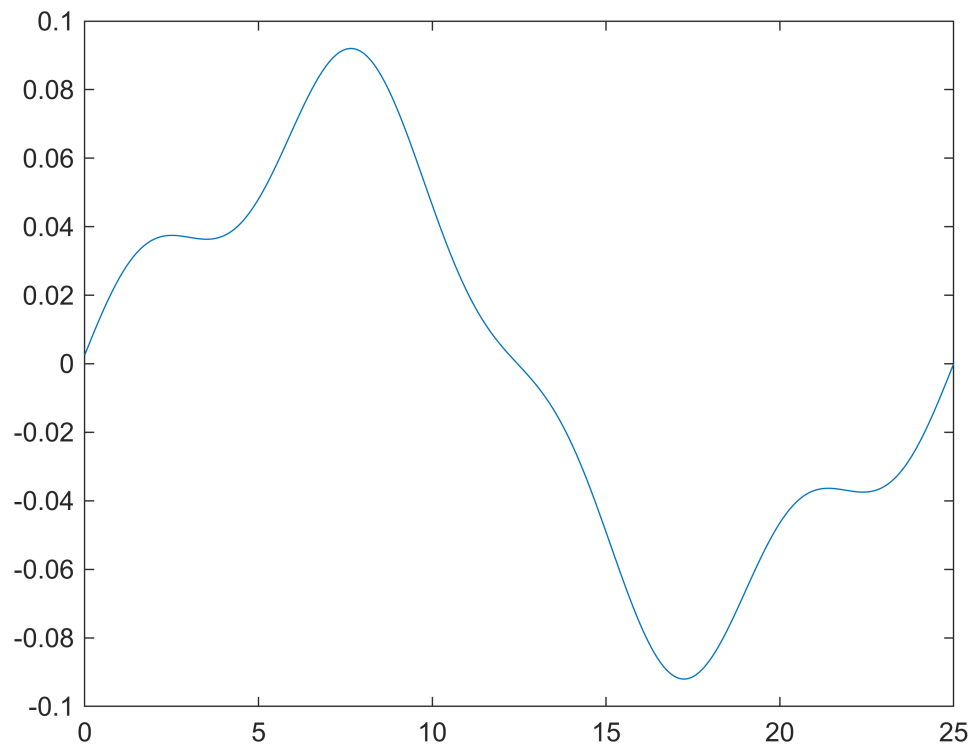
t=linspace(0,25,res);
t_vel=linspace(0,25-(1/res),res-1);

xvel_grad=gradient(x_coord,t);
yvel_grad=gradient(y_coord,t);

x_acc_grad=gradient(xvel_grad,t);
y_acc_grad=gradient(yvel_grad,t);

%plot(t,y_acc_grad)
%plot(t,yvel_grad)
plot(t,y_pos)

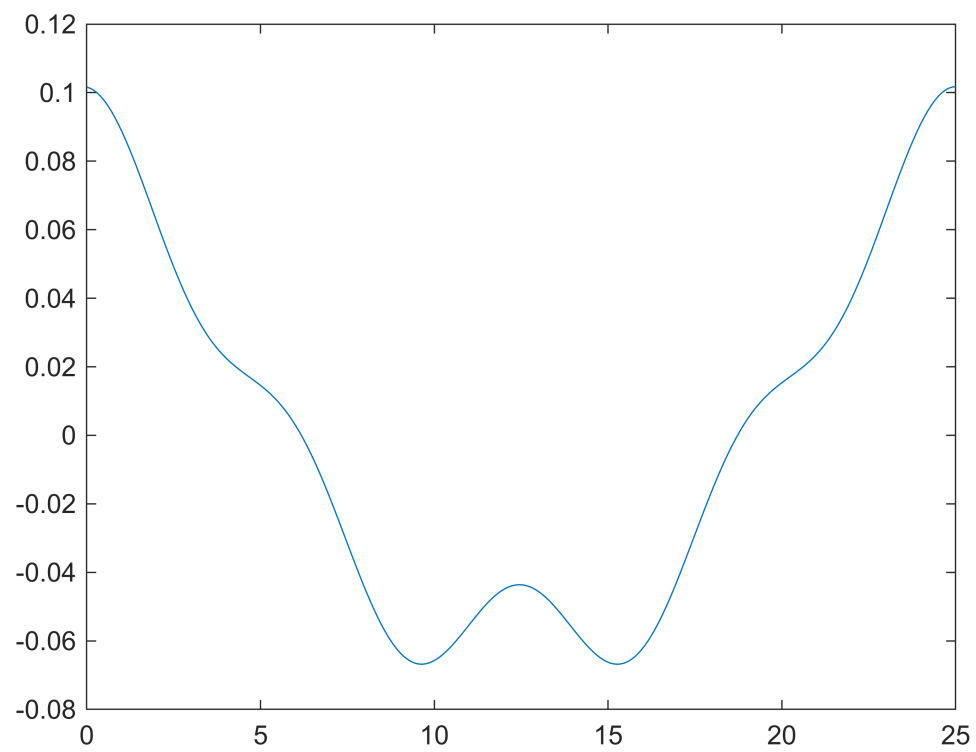
```



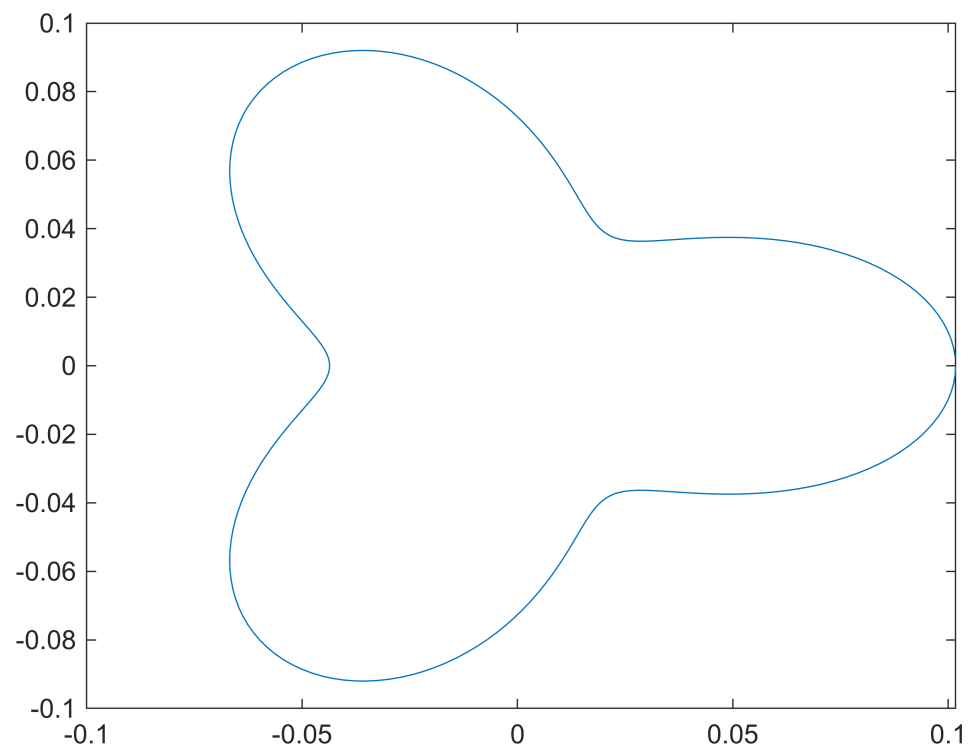
```

plot(t,x_pos);

```



```
clf;  
plot(x_pos,y_pos);
```



```
lengths=sqrt((x_pos+2).*(x_pos+2)+(y_pos+2).*(y_pos+2));
maxLenght=max(abs(lengths))
```

```
maxLenght =
2.9097
```

```
fun=@angArm
```

```
fun = function_handle with value:
    @angArm
```

```
x0=[0,0];
x=fsolve(fun,x0)
```

Warning: Trust-region-dogleg algorithm of FSOLVE cannot handle non-square systems; using Levenberg-Marquardt algorithm instead.
Equation solved.

fsolve completed because the vector of function values is near zero as measured by the value of the function tolerance, and the problem appears regular as measured by the gradient.

```
<stopping criteria details>
x = 1x2
10^-4 x
    0.7016    -0.4325
```

```
%Robot arm 2DOF
```

```
l1=0.22;
l2=0.17;
```

```
L(1) = Link([0,0,0, 0,0], 'modified');
L(1).offset=[0];
L(2) = Link([0,0,l1, 0,0], 'modified');
```

```
R = SerialLink(L, 'name', 'R', 'base', troty(180, 'deg'));
R.tool=[ [rotx(0);0 0 0], [l2;0;0;1] ];
```

```
r=sqrt(((y_pos(1)+22.5)^2)+(x_pos(1)+21)^2);
alpha=acos((l1^2 + r^2 - l2^2)/(2*20*r));
% D=((l1^2 + r^2 - l2^2)/(2*20*r));
% alpha=atan2(sqrt(1-D^2),D)
```

```
beta=asin((r/l2)*sin(alpha));
q1=atan((y_pos(1)+0.225)/(x_pos(1)+0.21))+alpha
```

```
q1 =
1.3207
```

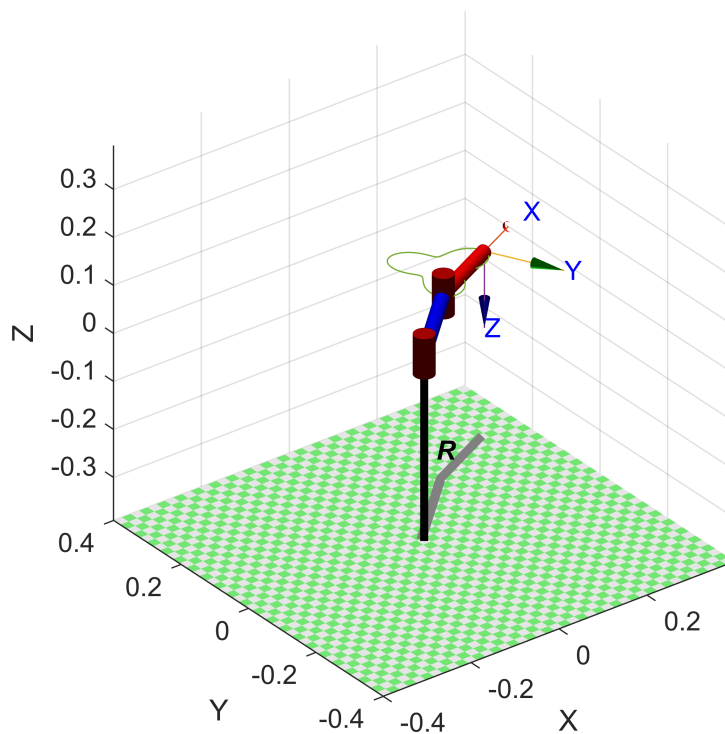
```
q2=beta-pi
```

```
q2 =  
-1.5708 - 5.4425i
```

```
poses_inicial=R.ikunc(transl((x_pos(1)+0.21),(y_pos(1)+0.225),0))
```

```
poses_inicial = 1x2  
2.3829 0.2938
```

```
clf;  
  
R.plot(poses_inicial)  
hold on  
plot(x_pos+0.21,y_pos+0.225)  
pbaspect([1 1 1])  
grid on
```



```
%posiciones angulares de cada articulacion  
q_positions=zeros(2,res);  
for i=1:length(t)  
    poses=R.ikunc(transl((x_pos(i)+0.21),(y_pos(i)+0.225),0));  
  
    q_positions(1,i)=poses(1);  
    q_positions(2,i)=poses(2)+pi/2;  
end
```

```

q_positions(1,77) = (q_positions(1,76)+q_positions(1,78))/2;
q_positions(2,77) = (q_positions(2,76)+q_positions(2,78))/2;
q_positions(1,101) = (q_positions(1,100)+q_positions(1,102))/2;
q_positions(2,101) = (q_positions(2,100)+q_positions(2,102))/2;

%% Trapezoidal

t=linspace(0,25+(0.1004*50),res+50);

Aproximacion= ctraj(R.fkine([2 2.6]),R.fkine(poses_inicial),50);
q_aprox_pos=[zeros(length(Aproximacion));zeros(length(Aproximacion))];
for i=1:length(Aproximacion)
    punto = R.ikunc(Aproximacion(i));
    q_aprox_pos(1,i)=punto(1);
    q_aprox_pos(2,i)=punto(2);
end

theta1=[q_aprox_pos(1,:) q_positions(1,:)];
theta2=[q_aprox_pos(2,:) q_positions(2,:)];

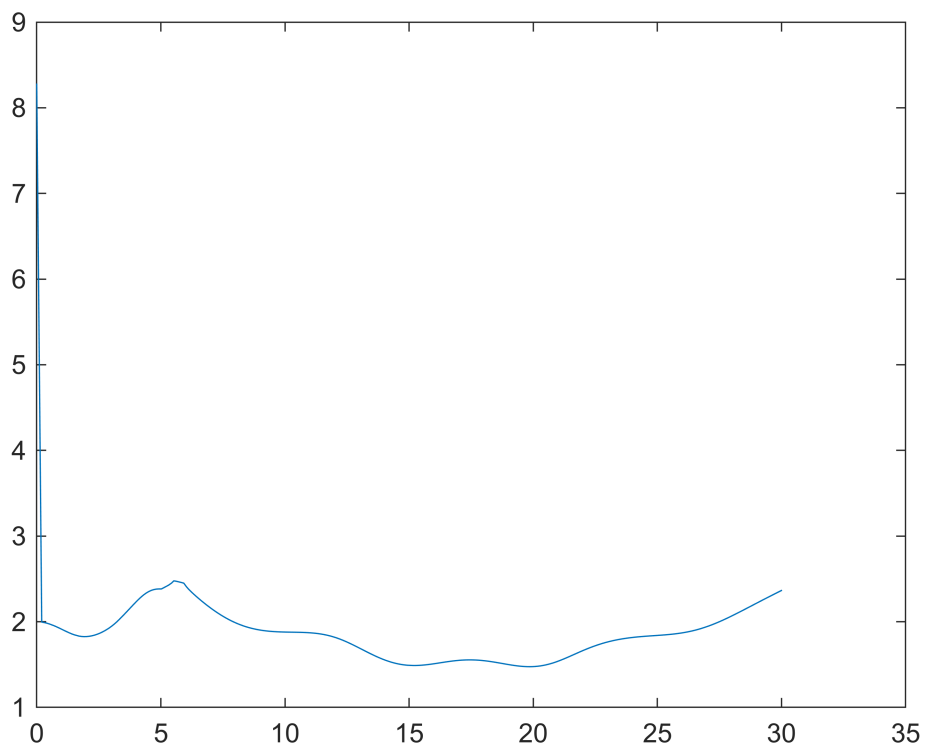
%Puntos arbitrarios para llegar, el punto 0= [29.1395 22.5051], la
%condicion de empezar se cumple en:

for i = 2:length(theta1)-1
    if abs(theta1(i) - theta1(i-1)) > 5
        theta1(i) = (theta1(i-1)+theta1(i+1))/2;
    end
end

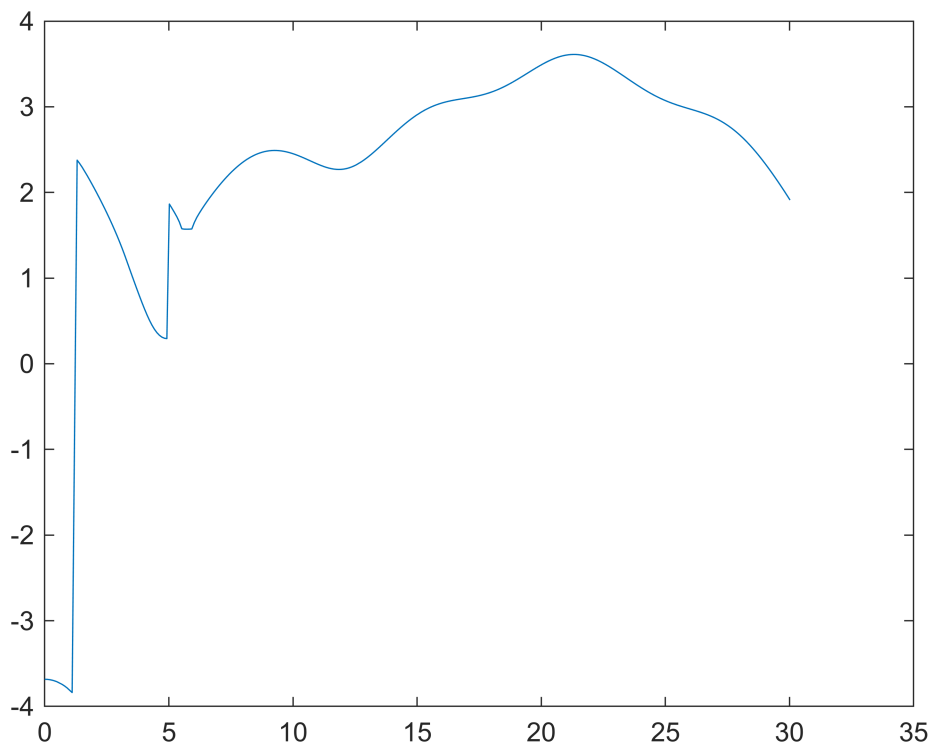
for i = 2:length(theta2)-1
    if abs(theta2(i) - theta2(i-1)) > 5
        theta2(i) = (theta2(i-1)+theta2(i+1))/2;
    end
end

%velocidad angular de cada eslabon
q1_angvel=gradient(theta1,t);
q2_angvel=gradient(theta2,t);
%aceleracion angular de cada eslabon
q1_angacc=gradient(q1_angvel,t);
q2_angacc=gradient(q2_angvel,t);
clf;
plot(t,theta1);

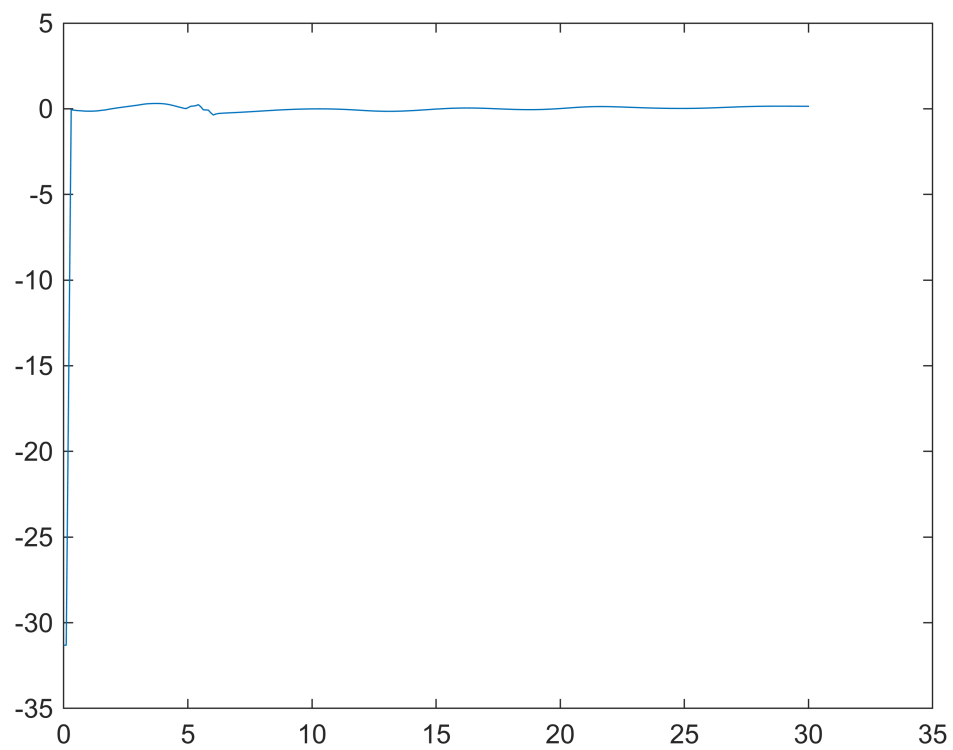
```



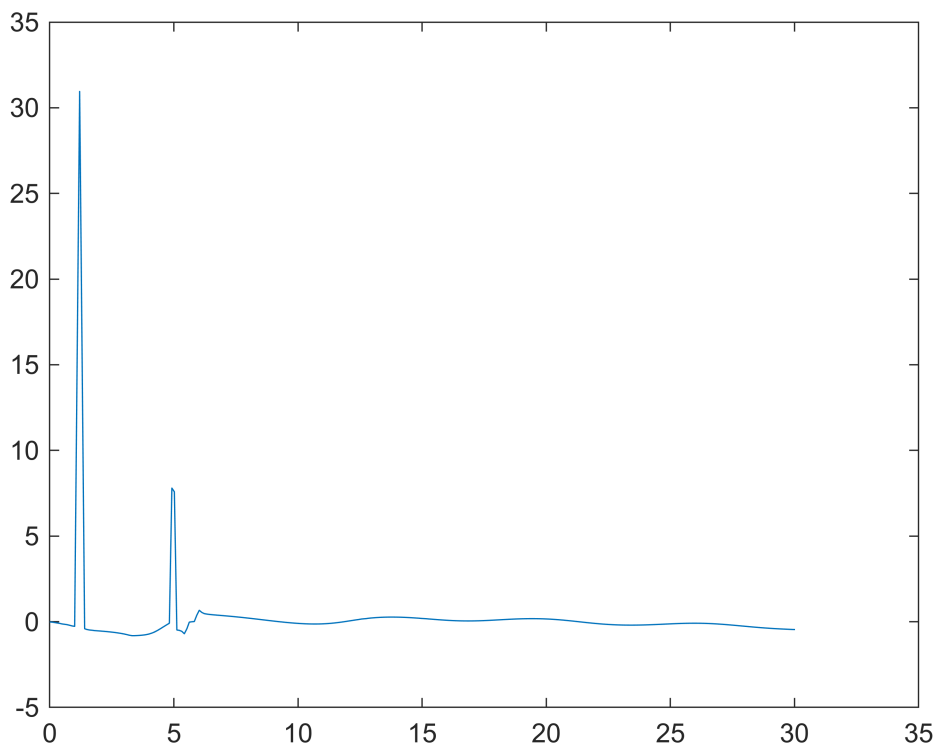
```
plot(t,theta2);
```




```
clf;  
plot(t,q1_angvel)
```



```
clf;  
plot(t,q2_angvel)
```



```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%Ahora hallamos la acc maxima para calcular el torque, JL es inercia
```

```
max(abs(q1_angacc))
```

```
ans =  
155.6294
```

```
max(abs(q2_angacc))
```

```
ans =  
156.1860
```

```
%Para hallar la masa se tiene que la densidad por metro del perfil T-slot  
%de alumionio es
```

```
M_density=0.52/100;
```

```
l1_mass=(l1)*M_density;
```

```
l2_mass=(l2)*M_density;
```

```
JL1=(1/12)*((l1/10)*M_density)*((l1/10)^2 + 0.002^2);
```

```
JL2=(1/12)*((l2/10)*M_density)*((l2/10)^2 + 0.002^2);
```

```
torque_inercial_max_q1=max(abs(q1_angacc))*JL1
```

```
torque_inercial_max_q1 =  
7.2403e-07
```

```
torque_inercial_max_q2=max(abs(q2_angacc))*JL2
```

```
torque_inercial_max_q2 =  
3.3712e-07
```

%%%

LAGRANGE

```
clf;
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%USING INTERNET FOUND INFO
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

M_density=0.52/1;
l1_mass=(l1)*M_density;
l2_mass=(l2)*M_density;
g=9.8;

theta_acc=[q1_angacc;q2_angacc];
theta1_vel = gradient(theta1,t);
theta2_vel = gradient(theta2,t);

% M_theta_q1=[( l1_mass*l1^2 + l2_mass*(l1^2 +
2*l1*l2*cos(q_positions(2,:))+l2^2)), l2_mass*(l1*l2*cos(q_positions(2,:))+l2^2) ;
( l2_mass*(l1*l2*cos(q_positions(2,:))+l2^2)) , (l2_mass*l2^2) ];
% c_theta=[ ( -l2_mass*l1*l2*sin(q_positions(2)).*( 2*
q1_angvel.*q2_angvel+q2_angvel.^2) ) ;
( l2_mass*l1*l2*q1_angvel.^2*sin(q_positions(2)) ) ];
% G=g*[ ( (l1_mass*l1+l2_mass*l1)*cos(q_positions(1))
+l2_mass*l2*cos(q_positions(1)+q_positions(2)) ) ;
( l2_mass*l2*cos(q_positions(1)+q_positions(2)) ) ]
%
% M_theta_q1=[( l1_mass*l1^2 + l2_mass*(l1^2 + 2*l1*l2.*cos(theta2)+l2^2)),
l2_mass*(l1*l2.*cos(theta2)+l2^2) ; ( l2_mass*(l1*l2.*cos(theta2)+l2^2)) ,
(l2_mass*l2^2) ];
% c_theta=[ ( -l2_mass*l1*l2*sin(theta2).*( 2*
q1_angvel.*q2_angvel+q2_angvel.^2) ) ; ( l2_mass*l1*l2*q1_angvel.^2*sin(theta2) ) ];
% G=g*[ ( (l1_mass*l1+l2_mass*l1)*cos(theta1)+l2_mass*l2*cos(theta1+theta2) ) ;
( l2_mass*l2*cos(theta1+theta2) ) ];
% Dynamic_torque=M_theta_q1*theta_acc+c_theta;

% %Q1 torque
% plot(t,abs(Dynamic_torque(1,:)))
% %Q2 torque
% plot(t,abs(Dynamic_torque(2,:)))
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%USING THE SLIDES FROM THE TEACHER
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
profile_side_lenght=0.002;
```

```
m1=(l1)*M_density;
```

```
m2=(l2)*M_density;
```

```
Iz1=(1/12)*((l1)*M_density)*((l1)^2 + profile_side_lenght^2);
```

```
Iz2=(1/12)*((l2)*M_density)*((l2)^2 + profile_side_lenght^2);
```

```
lc1=l1/2;
```

```
lc2=l2/2;
```

```
D_Lagrange=2*m2*l1*lc2*sin(theta2);
```

```
E_Lagrange=2*m2*l1*lc2*sin(theta2);
```

```
N_Lagrange=-m2*l1*lc2*sin(theta2);
```

```
M_theta_q1 =
```

```
mat2cell(repmat(zeros(2,2),1,length(theta1)),2,repmat(2,1,length(theta1)));
```

```
c_theta= mat2cell(repmat(zeros(2,1),1,length(theta1)),2,repmat(1,1,length(theta1)));
```

```
G= mat2cell(repmat(zeros(2,1),1,length(theta1)),2,repmat(1,1,length(theta1)));
```

```
Dynamic_torque=
```

```
mat2cell(repmat(zeros(2,1),1,length(theta1)),2,repmat(1,1,length(theta1)));
```

```
torque_q1=zeros(1,length(theta1));
```

```
torque_q2=zeros(1,length(theta1));
```

```
for i=1:length(theta1)
```

```
    M_theta_q1{i}=[ ( m1*lc1^2+Iz1+m2*l1^2 + m2*lc2^2 +Iz2+  
2*m2*l1*lc2.*cos(theta2(i)) ), ( m2*lc2^2 + Iz2 + m2*l1*lc2.*cos(theta2(i)) ) ;  
(m2*lc2^2 + m2*l1*lc2.*cos(theta2(i))+Iz2 ) , ( m2*lc2^2 +Iz2 ) ];
```

```
    c_theta{i}=[ ( D_Lagrange(i)*q1_angvel(i)* q2_angvel(i))  
+ E_Lagrange(i)*q2_angvel(i)^2 ; ( N_Lagrange(i)*theta1(i)*theta2(i)-(-  
m2*l1*lc2*sin(theta2(i))*theta1(i))*(theta1(i)+theta2(i)) ) ];
```

```
    G{i}=g*[ ( m1*lc1 + m2*l1 )*cos(theta1(i))+m2*lc2*cos(theta1(i)+theta2(i)) ;  
( m2*lc2*cos(theta1(i)+theta2(i)) ) ];
```

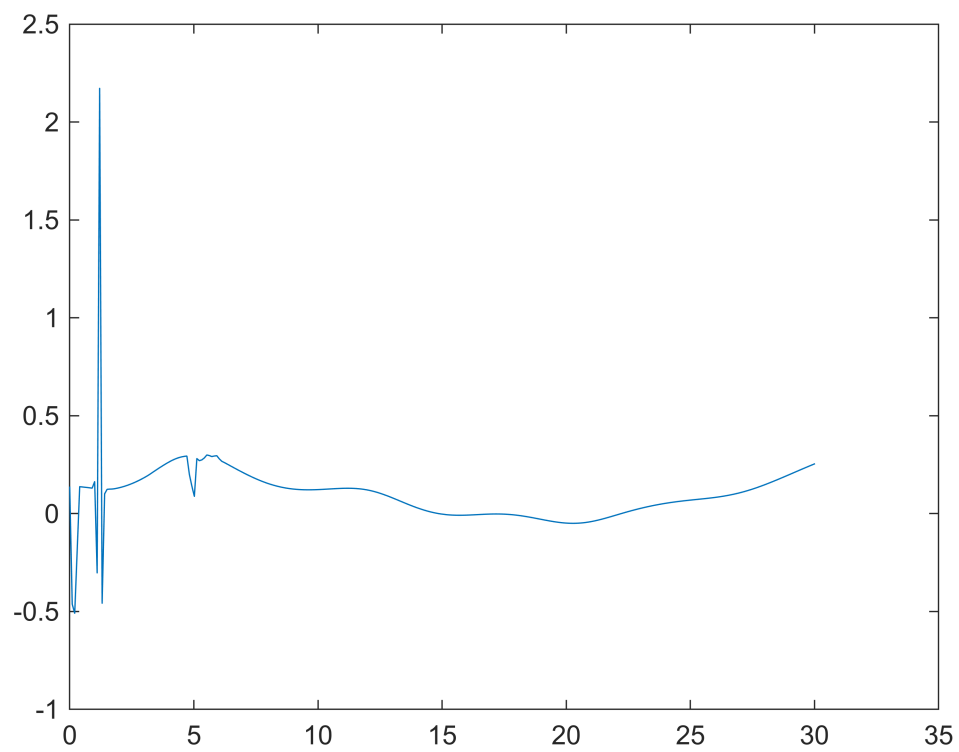
```
    Dynamic_torque{i}=M_theta_q1{i}*[q1_angacc(i);q2_angacc(i)] + c_theta{i} + G{i};
```

```
    torque_q1(i)=Dynamic_torque{i}(1);
```

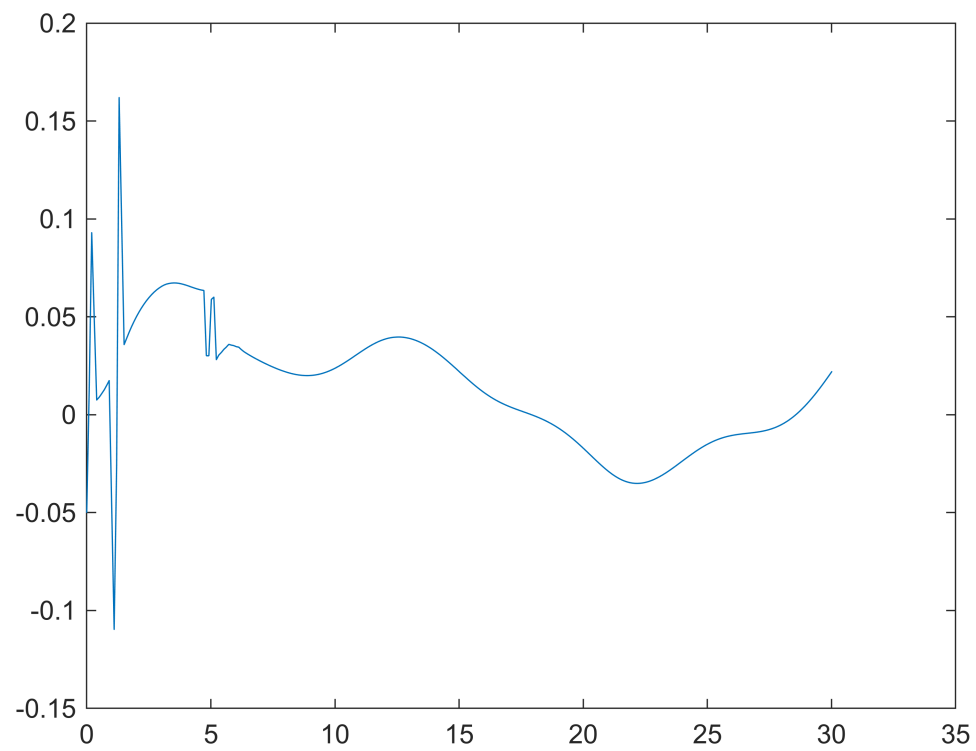
```
    torque_q2(i)=Dynamic_torque{i}(2);
```

```
end
```

```
plot(t,-torque_q1);
```



```
plot(t, -torque_q2);
```



```
q1_max_torque=max(abs(torque_q1))
```

```
q1_max_torque =  
2.1721
```

```
q1_rms_torque=rms(-torque_q1)
```

```
q1_rms_torque =  
0.1911
```

```
q1_max_vel=abs(max(q1_angvel))
```

```
q1_max_vel =  
0.3094
```

```
q2_max_torque=max(abs(torque_q2))
```

```
q2_max_torque =  
0.1620
```

```
q2_rms_torque=rms(-torque_q2)
```

```
q2_rms_torque =  
0.0335
```

```
q2_max_vel=abs(max(q2_angvel))
```

```
q2_max_vel =  
30.9548
```

SIMSCAPE

```
tm=1;  
km=1;  
Motor_G=tf([km],[tm 1 0])
```

```
Motor_G =
```

```
      1  
-----  
s^2 + s
```

Continuous-time transfer function.
Model Properties

```
function F=angArm(x,x_pos,y_pos)  
    F(1)=-0.5+1.2447*cos(x(1))+2*cos(x(1)-x(2));  
    F(1)=-3.1416e-04 +1.2447*sin(x(1))+2*sin(x(1)-x(2));  
end
```